TEMCO LEARN

Temco Platform

`temco-repositories` · built Nov 10, 2026

`temco-repositories`

The organisation and the local copies of it: the repository list, the working area, and reading a source into the platform's memory.

Folder	`extensions/temco-repositories`
Extension id	`temcocontrols.temco-repositories`
Namespace	`temcoRepositories`
State	active
Page	**The Repositories** (`temcoRepositories.page`) — a webview tab, opened by its own command
Settings	`temcoRepositories.organisation` , `temcoRepositories.workspaceFolder` , `temcoRepositories.projectsFolder`
Requires	`temco-store` (the platform's memory), `temco-ui` (where a page opens)
Required by	`temco-docs` — for the code files of a source and the material a new page is drafted from

Responsibilities, one file each

Module	Owns
`src/extension.ts`	activation, the page command, the output channel, the change signal, and the API
`src/repository.ts`	the organisation's repositories as an API read (paged, with a token when there is one), and a source's code files — from a working tree or from `git/trees`
`src/read.ts`	reading a source: the registry it declares, parsed for pages, recorded in the catalog with what the read concluded; the token in secret storage; opening a file over the API
`src/workarea.ts`	the working area: where a clone goes, whether one is there, how big it is, and clone / pull / remove
`src/repositoriesPage.ts`	the page: what the organisation has, what has been read, what has a local copy, and the buttons for each
`src/surface.ts`	the page's view type (`temcoRepositories.page`)
`src/store.ts` · `src/ui.ts`	what this module reads of `temco-store` and `temco-ui` — the copies of their shapes

Commands

`temcoRepositories.page` · `readSource` · `readAllSources` · `removeSource` · `revealSource` · `setSourceBranch` · `openFile` · `setToken` · `fetchSource` · `pullSource` · `openClone` · `removeClone` · `changeWorkspaceFolder` · `openWorkFolder` — the working area and reading, each with its own command rather than a mode.

Two of those are for other modules rather than for the palette, and are deliberately not declared: `projectWorkFolder` answers where a project's folder is and whether it exists, and `deleteProjectFolder` deletes it. `temco-projects` asks both when a project is removed, so the confirmation can name the folder before it goes — the working area is this module's, and the project row is that one's.

Boundary rules

- **Nothing is written to a repository, and nothing is ever pushed.** Every write this module makes is to the catalog; git is used for `clone` and `pull --ff-only` and for reading history, and never for a change to a tracked file.
- **The working area is outside every repository** (`temcoRepositories.workspaceFolder` , default `~/temco`), so a clone can never be mistaken for a source. A project's own folder is placed by a

second setting — `temcoRepositories.projectsFolder`, empty meaning "beside the clones" — so projects can be kept apart from the shared `source/` without moving it.

- **A token lives in secret storage**, keyed `temcoRepositories.githubToken`. It is never written to a setting, a file or the catalog.
- **Reading is explicit.** No read, clone or pull happens on its own: the first read offers a local copy, and every act after that is a command a person ran.
- **What a registry means is not decided here.** This module reads the file and records the rows; the registry's shape, its checks and its areas are `temco-docs`', which is why this module needs nothing from it.
- **The page opens where our pages open.** That rule is `temco-ui`'s — one group, never a split — and it is read here rather than re-decided (`src/ui.ts`).